ERO | PRATIQUE | SUBA DE NIVEL

G+

MISSAO ALGORITMO

Logica de Programacao Gamificada com VisualG

Uma apostila gratuita, passo a passo, para dar os primeiros passos em logica de programacao com VisuAlg.

SEM PRE-REQUISITOS	0 PAGINAS	PORTUGOL / VISUALG

PRÓLOGO · 01

Bem-vinda à sua primeira missão

Esta apostila é para quem olha para uma tela de código e pensa: 'não sei nem por onde começar'. Você não precisa ter estudado matemática avançada, conhecer computadores por dentro ou já ter escrito uma linha de código. Vamos construir o entendimento em camadas: primeiro o problema e o raciocínio; depois os passos; por fim, a escrita e a execução no VisuAlg. Leia sem pressa, teste os exemplos e volte às páginas anteriores sempre que precisar.

O que você vai aprender

Entender computador, programa, lógica, algoritmo, pseudocódigo e IDE; planejar soluções; escrever e testar algoritmos no VisuAlg.

Ao concluir

Você será capaz de explicar a diferença entre lógica, algoritmo e programa; ler e escrever soluções simples; usar variáveis, entradas e saídas, decisões, repetições, procedimentos, funções e vetores; testar e corrigir erros comuns.

Como a trilha avança

Começamos com conceitos do cotidiano e exemplos sem código. Em seguida desenhamos soluções, conhecemos a ferramenta e escrevemos programas cada vez mais completos, até montar jogos e um projeto final.

Como estudar

Faça uma missão por vez. Antes de ver a pista, tente responder. Simule os valores no papel, digite o exemplo, execute, mude uma coisa e observe o que aconteceu. Errar é uma informação útil para a próxima tentativa.

O que você precisa

Curiosidade, papel e lápis. O VisuAlg é apresentado adiante; você pode ler a introdução primeiro e instalar ou abrir a ferramenta quando chegar à missão de preparação.

DESAFIO **+10 XP**

Escreva o que gostaria de conseguir criar com programação. No fim da apostila, volte a esta resposta e veja o quanto avançou.

PISTA / GABARITO Não tente decorar tudo. Entenda uma ideia, experimente e explique-a com suas próprias palavras.

MUNDO 1 · 03

O que é um computador?

Um computador recebe informações, guarda parte delas, executa instruções e apresenta resultados. Ele não adivinha a intenção de quem programa: precisa de comandos claros, organizados e possíveis de executar.

Entrada

Dados que chegam ao sistema: teclado, mouse, sensor, arquivo ou uma resposta digitada.

Processamento

Instruções transformam ou analisam os dados. A CPU executa operações seguindo a ordem definida.

Saída e armazenamento

A tela, o som ou uma impressão mostram resultados; memória e disco guardam dados por tempos diferentes.

TERMINAL VISUALG — MISSAO 03

```
ENTRADA: duas notas
PROCESSAMENTO: calcular a média
SAÍDA: mostrar a média
```

DESAFIO — +15 XP

Para um despertador, identifique uma entrada, um processamento e uma saída.

PISTA / GABARITO Exemplo: hora configurada; comparar com o relógio; tocar o alarme.

MUNDO 1 · 04

Hardware, software e dados

Hardware é a parte física que você pode tocar. Software é o conjunto de instruções e programas. Dados são as informações que esses programas recebem, produzem ou armazenam.

Hardware
Teclado, monitor, memória, processador e celular são componentes físicos.

Software
Sistema operacional, navegador, editor e jogo orientam o hardware a realizar tarefas.

Dados
Um nome, uma nota ou uma temperatura não é o programa: é informação processada pelo programa.

DESAFIO +15 XP

Classifique: navegador, mouse, senha digitada e resultado exibido na tela.

PISTA / GABARITO Software, hardware, dado de entrada e dado de saída.

MUNDO 1 · 05

Programar, linguagem e IDE

Programar é transformar uma solução em instruções que uma ferramenta consegue interpretar e executar. Para isso usamos uma linguagem com palavras e regras. E escrevemos dentro de um ambiente que reúne ferramentas úteis para criar e testar o programa.

Lógica e linguagem

A lógica ajuda a decidir como resolver o problema. A linguagem serve para registrar os passos com uma sintaxe específica. A mesma lógica pode ser escrita em linguagens diferentes.

O que é uma IDE?

IDE significa Ambiente de Desenvolvimento Integrado. É um programa que reúne recursos como editor de código, execução e mensagens de erro; algumas IDEs também oferecem ferramentas para acompanhar o programa enquanto ele roda.

Editor e interpretador

O editor é a área em que escrevemos. O interpretador lê o pseudocódigo, verifica se a forma está correta e executa as instruções entendidas. A mensagem de erro é uma pista sobre algo que precisa ser revisto.

Por que usar uma ferramenta didática?

No começo, muitos detalhes técnicos podem distrair do raciocínio. Uma ferramenta educacional permite praticar a sequência, as decisões e os dados com palavras familiares, antes de avançar para linguagens e ferramentas profissionais.

TERMINAL VISUALG — **MISSAO 05**

```
IDE = editor + execução + apoio para desenvolver
LÓGICA = raciocínio para resolver
LINGUAGEM = forma de escrever a solução
```

DESAFIO — **+20 XP**

Na sua rotina, qual ferramenta pode ser considerada uma IDE? Pense em uma que reúna editor e recursos para executar ou testar o que você escreve.

PISTA / GABARITO

Uma IDE não é uma linguagem de programação. Ela é o ambiente que ajuda a criar programas escritos em uma linguagem.

MUNDO 1 · 06

Algoritmo: um plano executável

Algoritmo é uma sequência finita e organizada de passos para resolver um problema ou alcançar um resultado. A receita culinária é uma analogia útil, mas um algoritmo também trata cálculos, decisões e repetições.

Tem objetivo

Antes de escrever passos, diga qual resultado você quer obter e quais dados estão disponíveis.

Tem ordem

Trocar a ordem pode mudar o resultado: misturar ingredientes antes de medir pode alterar a receita.

Termina

Uma solução precisa ter um caminho de conclusão. Repetir para sempre costuma ser um defeito, não uma estratégia.

DESAFIO **+20 XP**

Escreva um algoritmo de cinco passos para calcular quanto troco falta após uma compra.

PISTA / GABARITO Entrada: preço e valor pago. Processo: subtrair. Saída: mostrar o troco.

MUNDO 1 · 07

Lógica, algoritmo e programa

Lógica de programação é a forma estruturada de raciocinar sobre problemas. Algoritmo é uma solução passo a passo. Programa é essa solução escrita numa linguagem executável, como o pseudocódigo do VisuAlg.

Lógica
Escolher relações, regras e ordem para chegar a uma resposta correta.

Algoritmo
Descrever os passos da solução sem depender de uma linguagem específica.

Programa
Codificar os passos com sintaxe e comandos de uma linguagem.

DESAFIO **+20 XP**

Diga se cada item é lógica, algoritmo ou programa: a regra "se chove, leve guarda-chuva"; a lista ordenada de passos; o código VisuAlg.

PISTA / GABARITO Regra de raciocínio, algoritmo descrito, programa escrito.

MUNDO 1 · 08

Quebre o problema em partes

Problemas grandes assustam porque misturam várias tarefas. Decompor é separar o objetivo em partes pequenas, descobrir o que cada parte precisa e combinar os resultados.

Defina a missão
Use uma frase: “Quero calcular o preço final com desconto”. Evite começar pelo código.

Separe subtarefas
Ler o preço, ler o percentual, calcular o desconto, subtrair e mostrar o total.

Procure padrões
Se vários produtos usam o mesmo cálculo, existe uma regra que pode ser reaproveitada.

DESAFIO +20 XP

Decomponha “organizar uma viagem” em quatro subtarefas que um app poderia ajudar a controlar.

PISTA / GABARITO Datas, destino, orçamento e checklist são bons blocos iniciais.

MUNDO 1 · 09

Entrada, processo e saída

O modelo IPO (Input, Process, Output) ajuda a enxergar os limites de uma solução. Em português, pense em entrada, processamento e saída. Primeiro identifique o que chega, depois o que acontece e por fim o que deve aparecer.

Entrada
Dados necessários. Se falta uma entrada, o processo não tem informação suficiente.

Processo
Fórmula, regra, decisão ou sequência que transforma as entradas.

Saída
Resultado que responde à pergunta do problema, com unidade e rótulo claros.

TERMINAL VISUALG — MISSAO 09

```
LEIA: largura, altura
CALCULE: area <- largura * altura
MOSTRE: area em cm2
```

DESAFIO — +20 XP

Desenhe o IPO de um conversor de minutos para horas e minutos.

PISTA / GABARITO — Entrada: minutos totais. Processo: divisão e resto. Saída: horas e minutos restantes.

MUNDO 1 · 10

Fluxograma: desenhe o caminho

Fluxograma representa o fluxo de um algoritmo com formas conectadas por setas. Ele é útil quando uma solução tem decisões ou caminhos alternativos e ajuda a conversar sobre a lógica antes de programar.

Oval
Marca início ou fim.

Retângulo
Representa uma ação ou cálculo.

Paralelogramo
Representa leitura ou exibição de dados.

Losango
Faz uma pergunta lógica e abre caminhos como SIM e NÃO.

DESAFIO +20 XP

Desenhe um fluxograma para decidir se uma pessoa pode entrar: leia idade, compare com 18 e mostre permitido ou não permitido.

PISTA / GABARITO Use início -> entrada -> losango -> dois resultados -> fim.

MUNDO 1 · 11

VisuAlg: seu laboratório de algoritmos

VisuAlg é uma ferramenta didática que permite escrever, executar e acompanhar algoritmos em pseudocódigo, uma forma estruturada de descrever programas com palavras próximas do português. Ele funciona como um laboratório: você formula uma ideia, dá instruções ao computador e observa o resultado.

Por que começar com VisuAlg?

A sintaxe em português estruturado reduz a barreira inicial e deixa o foco nos fundamentos: ordem, variáveis, escolhas e repetições. Você pratica a lógica sem precisar começar memorizando muitas regras de uma linguagem profissional.

O que ele não é

VisuAlg é ótimo para aprender os primeiros conceitos, mas é uma ferramenta educacional. Conhecê-lo não significa que você já domina todas as tecnologias usadas em empresas; significa que está construindo uma base que depois poderá transferir para outras linguagens.

Seu primeiro ciclo

Abra ou crie um algoritmo, escreva as instruções, salve o arquivo, execute e leia a saída. Depois teste com outros valores. Se houver erro, leia a mensagem, confira a linha indicada e revise nomes, palavras reservadas, parênteses e a ordem dos comandos.

Como estudar nesta apostila

Leia o exemplo antes de digitar e tente prever o resultado. Execute, compare com sua previsão e explique a diferença. Salve cada exercício com um nome que lembre a tarefa; assim você consegue voltar e melhorar suas soluções.

Compatibilidade

Há versões e interpretadores compatíveis com diferenças de interface. Procure uma distribuição confiável para seu sistema e acompanhe as instruções dela. Os exemplos usam a sintaxe clássica ensinada nas próximas páginas.

DESAFIO **+20 XP**

Prepare seu espaço de estudo: escolha onde vai executar os exemplos e crie uma pasta chamada missao_algoritmo para guardar seus exercícios.

PISTA / GABARITO

Ainda não precisa instalar nada para entender os conceitos. Quando abrir a ferramenta, a missão seguinte será executar seu primeiro programa.

MUNDO 1 · 12

Seu primeiro programa

O primeiro programa deve provar que a ferramenta executa uma instrução. `escreval` imprime um texto e termina a linha. O programa inteiro precisa de começo, área de comandos e encerramento.

Nome
A primeira linha identifica o algoritmo entre aspas.

Corpo
Depois de inicio ficam os comandos que serão executados.

Fim
fimalgoritmo encerra o pseudocódigo.

TERMINAL VISUALG — MISSAO 12

```
algoritmo "ola_exploradora"
inicio
escreval("Ola, mundo!")
fimalgoritmo
```

DESAFIO — +25 XP

Troque a mensagem por seu nome de explorador e por uma frase de boas-vindas.

PISTA / GABARITO — Acentos dentro do texto entre aspas são conteúdo; palavras reservadas ficam sem acento.

MUNDO 2 · 13

A estrutura do algoritmo

No VisuAlg, a declaração de variáveis vem antes de `inicio`. O corpo principal fica entre `inicio` e `fimalgoritmo`. Comentários começam com `//` e ajudam pessoas a entender a intenção.

Cabeçalho
algoritmo seguido de nome entre aspas.

Declarações
var apresenta as variáveis que o algoritmo usará.

Comandos
A execução do programa principal começa em inicio e segue linha a linha.

TERMINAL VISUALG — MISSAO 13

```
algoritmo "mensagem"
var nome: caractere
inicio
// A pessoa informa o nome
escreva("Ola, ", nome)
fimalgoritmo
```

DESAFIO — +20 XP

Aponte no exemplo cabeçalho, declaração, comentário e comando.

PISTA / GABARITO — A variável ainda precisa receber um valor antes de ser usada.

MUNDO 2 · 14

Variáveis: caixas com nome

Uma variável é um espaço identificado pelo nome, cujo conteúdo pode ser consultado e atualizado durante a execução. Imagine uma caixa etiquetada: o nome aponta para o dado atual, não para todos os dados que ela já guardou.

Nome útil
Use nota_final em vez de x quando o propósito for uma nota.

Valor muda
Depois de uma nova atribuição, a variável passa a representar o valor mais recente.

Estado
O conjunto dos valores atuais das variáveis é o estado do algoritmo naquele momento.

TERMINAL VISUALG — MISSAO 14

```
pontos <- 5
pontos <- pontos + 3
escreval(pontos) // 8
```

DESAFIO — +20 XP

Rastreie `vidas <- 3`, depois `vidas <- vidas - 1`. Qual é o valor final?

PISTA / GABARITO — A segunda linha lê o valor antigo e grava o resultado novo: 2.

MUNDO 2 · 15

Tipo inteiro

O tipo `inteiro` guarda números sem parte decimal, como 0, 12 ou -4. Escolha-o para quantidades discretas, contagens, idade em anos completos ou número de tentativas.

Contar

Uma quantidade de itens costuma ser inteira: 7 livros, 3 vidas, 12 rodadas.

Limites

Uma variável tem capacidade finita. Evite esperar que um tipo aceite qualquer número imaginável.

Cuidado

Uma divisão de inteiros pode ter resultado com parte decimal; decida se precisa de tipo real.

TERMINAL VISUALG — MISSAO 15

```
var vidas, fase: inteiro
inicio
vidas <- 3
fase <- 1
```

DESAFIO — +15 XP

Escolha entre inteiro ou real para: quantidade de alunos, altura em metros, número de gols.

PISTA / GABARITO — Inteiro, real, inteiro.

MUNDO 2 · 16

Tipo real

O tipo `real` representa números que podem ter casas decimais, como 1.75 ou 19.90. No código do VisuAlg, o separador decimal é ponto; na conversa e na exibição brasileira, costuma-se falar com vírgula.

Medidas
Altura, distância, temperatura e preço geralmente precisam aceitar frações.

Atenção
Representações numéricas no computador podem ter pequenas aproximações. Para dinheiro real, sistemas comerciais usam centavos inteiros ou tipos decimais apropriados.

Teste
Use entradas com fração e valores inteiros para confirmar os dois caminhos.

TERMINAL VISUALG — MISSAO 16

```
var preco, desconto: real
inicio
preco <- 39.90
desconto <- preco * 0.10
```

DESAFIO — +20 XP

Declare uma variável para temperatura e calcule uma elevação de 2.5 graus.

PISTA / GABARITO — Use real nas duas variáveis; some 2.5 à temperatura.

MUNDO 2 · 17

Tipo caractere

O tipo `caractere` guarda texto: nomes, respostas, códigos e mensagens. Texto literal no VisuAlg fica entre aspas duplas. Evite misturar texto e número sem decidir como a operação deve funcionar.

Texto

Uma palavra ou frase pode ser guardada em uma variável caractere.

Comparação

Textos podem ser comparados, mas diferenças de maiúsculas, acentos e espaços podem afetar o resultado.

Entrada

Leia o texto em uma variável antes de usá-lo em mensagem ou condição.

TERMINAL VISUALG — MISSAO 17

```
var jogador: caractere
inicio
escreva("Seu nome: " )
leia(jogador)
escreval("Bem-vinda, ", jogador)
```

DESAFIO — +20 XP

Crie um algoritmo que leia o nome de um planeta favorito e o mostre numa frase.

PISTA / GABARITO — Use jogador: caractere, leia(jogador) e escreval.

MUNDO 2 · 18

Tipo lógico

Uma variável lógica guarda apenas dois estados: `verdadeiro` ou `falso`. Ela é ótima para representar respostas de sim/não, condições atendidas e sinalizadores de controle.

Dois valores
Não existe um terceiro estado lógico básico nessa declaração.

Perguntas
Uma comparação como idade >= 18 produz um resultado lógico.

Decisões
O comando se usa esse resultado para escolher qual bloco executar.

TERMINAL VISUALG — MISSAO 18

```
var tem_chave: logico
inicio
tem_chave <- verdadeiro
escreval(tem_chave)
```

DESAFIO — +20 XP

Defina uma variável lógica para "missão concluída" e atribua falso inicialmente.

PISTA / GABARITO — Use missao_concluida: logico e missao_concluida <- falso.

MUNDO 2 · 19

Declaração e atribuição

Declarar informa o nome e o tipo antes do uso. Atribuir coloca um valor na variável. No VisuAlg a seta `<-` lê-se “recebe”; não é uma igualdade matemática.

Declarar
Em `var idade: inteiro`, a variável passa a existir com tipo conhecido.

Atribuir
Em `idade <- 20`, o valor 20 é guardado em idade.

Atualizar
Em `idade <- idade + 1`, o lado direito é calculado antes de atualizar o esquerdo.

TERMINAL VISUALG — MISSAO 19

```
var energia: inteiro
inicio
energia <- 10
energia <- energia - 4
```

DESAFIO — +20 XP

Se saldo começa em 30 e recebe `saldo <- saldo + 12`, qual é o novo saldo?

PISTA / GABARITO — 42. A seta não pergunta se saldo é igual à soma; ela atualiza.

MUNDO 2 · 20

Leia e escreva dados

`leia` recebe dados digitados. `escreva` imprime sem avançar a linha; `escreval` imprime e avança. A entrada precisa combinar com o tipo da variável que vai recebê-la.

Pergunte
Mostre a pergunta antes de usar leia para orientar a pessoa.

Leia
Guarde a resposta numa variável declarada com tipo adequado.

Responda
Exiba uma frase clara e inclua unidade ou contexto quando necessário.

TERMINAL VISUALG — MISSAO 20

```
algoritmo "dobro"
var n: real
inicio
escreva("Digite um numero: " )
leia(n)
escreval("Dobro: ", n * 2)
fimalgoritmo
```

DESAFIO — +25 XP

Adapte o programa para pedir um nome e uma cor favorita e imprimir ambos.

PISTA / GABARITO — Use duas variáveis caractere e dois comandos leia.

MUNDO 2 · 21

Operadores matemáticos

Operadores combinam valores para produzir resultados. No VisuAlg, use `+`, `-`, `*`, `/`, `div` e `mod` conforme o tipo de cálculo. Verifique sempre se o resultado precisa manter casas decimais.

Aritmética

Adição, subtração, multiplicação e divisão usam +, -, * e /.

Quociente inteiro

`div` obtém a divisão inteira em situações compatíveis.

Resto

`mod` obtém o resto e ajuda a detectar pares, ciclos e unidades restantes.

TERMINAL VISUALG — MISSAO 21

```
escreval(17 div 5) // 3
escreval(17 mod 5) // 2
escreval(17 / 5) // 3.4
```

DESAFIO — +20 XP

Quantas caixas completas de 6 cabem em 29 itens? Quantos itens sobram?

PISTA / GABARITO — 29 div 6 = 4; 29 mod 6 = 5.

MUNDO 2 · 22

Prioridade e parênteses

Expressões seguem uma ordem de operações. Parênteses tornam essa ordem visível e reduzem surpresas. Ao traduzir uma fórmula, preserve primeiro a estrutura matemática e só depois simplifique.

Multiplicar antes

Multiplicação e divisão acontecem antes de adição e subtração, salvo parênteses.

Agrupe

Escreva `(a + b) / 2` para média, e não `a + b / 2`.

Leia em etapas

Use variáveis intermediárias para tornar cálculos longos mais fáceis de conferir.

TERMINAL VISUALG — MISSAO 22

```
media <- (nota1 + nota2) / 2
area <- base * altura
total <- preco - (preco * taxa / 100)
```

DESAFIO — +25 XP

*Com a=8, b=4 e c=2, compare a+b*c com (a+b)*c.*

PISTA / GABARITO — Primeiro dá 16; segundo dá 24. Os parênteses mudam a rota.

MUNDO 2 · 23

Comparações viram perguntas

Operadores relacionais comparam valores e produzem verdadeiro ou falso. As respostas podem controlar decisões e repetições. Não confunda atribuição `<-` com igualdade `=`.

Maior e menor

`>`, `<`, `>=` e `<=` comparam magnitudes.

Igual e diferente

`=` testa igualdade; `<>` testa diferença no VisuAlg.

Resultado lógico

A expressão `tentativas < 3` pode ser verdadeira ou falsa.

TERMINAL VISUALG — MISSAO 23

```
idade >= 18
senha <> ""
nota = 10
```

DESAFIO — +20 XP

Para `energia=4`, calcule `energia > 5`, `energia <= 4` e `energia = 0`.

PISTA / GABARITO — Falso, verdadeiro, falso.

MUNDO 2 · 24

E, ou, não

Operadores lógicos combinam condições. `e` exige que todas sejam verdadeiras; `ou` exige pelo menos uma; `nao` inverte o resultado. Use parênteses para deixar a intenção explícita.

E
Acesso liberado pode exigir `senha_correta e cadastro_ativo`.

Ou
Um desconto pode valer se `cupom_valido ou cliente_vip`.

Não
`nao tem_erro` representa o oposto lógico de `tem_erro`.

TERMINAL VISUALG — MISSAO 24

```
se idade >= 18 e tem_ingresso entao
escreval("Entrada liberada")
fimse
```

DESAFIO — +25 XP

Acesso infantil: idade <= 12 e acompanhado. A pessoa tem 10 anos e está acompanhada?

PISTA / GABARITO — As duas partes são verdadeiras, então o acesso é liberado.

MUNDO 2 · 25

Decisão com se

Uma condição escolhe se um bloco será executado. Em VisuAlg, `se ... entao` abre a decisão e `fimse` fecha o bloco. O algoritmo continua depois do bloco, quer a condição seja verdadeira ou falsa.

Pergunta
A expressão precisa produzir verdadeiro ou falso.

Bloco
Indente os comandos para enxergar o que pertence à decisão.

Teste
Experimente uma entrada que torna a condição verdadeira e outra que a torna falsa.

TERMINAL VISUALG — MISSAO 25

```
se pontos >= 100 entao
escreval("Nova fase!")
fimse
```

DESAFIO — +25 XP

Mostre “meta atingida” se vendas for igual ou maior que 10.

PISTA / GABARITO — Use `se vendas >= 10 entao` e feche com `fimse`.

MUNDO 2 · 26

Se e senão

Quando há dois caminhos, `senao` define a alternativa para quando a condição é falsa. Cada caminho pode ter mais de um comando, mas os dois ficam dentro do mesmo `fimse`.

Verdadeiro
Executa os comandos antes de senao.

Falso
Executa os comandos depois de senao.

Exclusivos
Em uma decisão simples, apenas um dos dois blocos é executado.

TERMINAL VISUALG — MISSAO 26

```
se nota >= 7 entao
escreval("Aprovada")
senao
escreval("Precisa revisar")
fimse
```

DESAFIO — +25 XP

Decida se temperatura está abaixo de zero. Exiba “congelante” ou “acima de zero”.

PISTA / GABARITO — Teste também zero; a condição < 0 o coloca no segundo caminho.

MUNDO 2 · 27

Várias opções: escolha

Aninhar muitos `se` pode dificultar a leitura. Quando uma única variável precisa ser comparada com várias opções discretas, `escolha`, `caso`, `outrocaso` e `fimescolha` deixam os caminhos organizados.

Seleção
A expressão de escolha é comparada com os valores listados em cada caso.

Padrão
outrocaso cobre uma opção inesperada ou não listada.

Uso certo
Escolha serve para opções exatas; faixas como nota maior que 7 continuam boas candidatas a se.

TERMINAL VISUALG — MISSAO 27

```
escolha menu
caso 1
escreval("Iniciar")
caso 2
escreval("Ajuda")
outrocaso
escreval("Opcao invalida")
fimescolha
```

DESAFIO — +25 XP

Crie opções para 1=água, 2=suco e demais=opção inválida.

PISTA / GABARITO — Leia um inteiro menu antes da estrutura escolha.

MUNDO 2 · 28

Valide antes de usar

Validação verifica se os dados fazem sentido para a regra do problema. O objetivo não é desconfiar da pessoa, mas impedir que uma entrada inválida conduza a um resultado sem significado.

Faixa
Idade não deve ser negativa; nota pode estar entre 0 e 10.

Campo obrigatório
Um nome vazio pode impedir uma mensagem personalizada.

Regra
Descreva o que é válido em linguagem simples e teste limite, abaixo do limite e acima.

TERMINAL VISUALG — MISSAO 28

```
se nota < 0 ou nota > 10 entao
escreval("Nota invalida")
senao
escreval("Nota recebida")
fimse
```

DESAFIO — +30 XP

Que valores testaria para validar uma nota de 0 a 10?

PISTA / GABARITO — 0 e 10, um dentro da faixa (5), um abaixo (-1) e um acima (11).

MUNDO 3 · 29

Repetição: automatize o que se repete

Laços repetem um bloco sem copiar e colar as mesmas instruções. A escolha depende da pergunta: você sabe quantas vezes vai repetir, repete enquanto uma condição valer ou precisa executar pelo menos uma vez?

Para
Use quando a quantidade de repetições é conhecida ou controlada por contagem.

Enquanto
Use quando continuar depende de uma condição testada antes do bloco.

Repita
Use quando o bloco deve acontecer pelo menos uma vez e a condição de parada é testada no final.

DESAFIO +20 XP

Qual laço combina melhor com: exibir 1 a 10; esperar senha correta; perguntar se quer jogar de novo?

PISTA / GABARITO Para; enquanto; repita.

MUNDO 3 · 30

Laço para

`para` reúne contador, início, limite e passo numa estrutura. É excelente para percorrer uma faixa conhecida, como níveis, dias ou posições de um vetor.

Início
A variável de controle começa no valor indicado.

Limite
A execução segue até o limite, considerando o sentido do passo.

Passo
`passo 1` avança; passo negativo pode percorrer na direção contrária quando a faixa está definida para isso.

TERMINAL VISUALG — MISSAO 30

```
para fase de 1 ate 5 faca
escreval("Fase ", fase)
fimpara
```

DESAFIO — +25 XP

Altere o laço para mostrar números de 2 até 12, de 2 em 2.

PISTA / GABARITO — Use `passo 2`.

MUNDO 3 · 31

Laço enquanto

`enquanto ... faca` verifica a condição antes de cada volta. Se ela já começa falsa, o bloco não executa nenhuma vez. Por isso, prepare as variáveis antes do laço e garanta uma mudança que possa encerrar a repetição.

Inicialização
Defina o contador ou estado antes de enquanto.

Condição
Escreva claramente o que permite continuar.

Atualização
Mude algo dentro do laço para se aproximar da parada.

TERMINAL VISUALG — MISSAO 31

```
contador <- 1
enquanto contador <= 3 faca
escreval(contador)
contador <- contador + 1
fimenquanto
```

DESAFIO — +30 XP

Mostre 5, 4, 3, 2, 1 usando enquanto.

PISTA / GABARITO Comece em 5; continue enquanto > 0; subtraia 1.

MUNDO 3 · 32

Laço repita até

`repita ... ate` executa o bloco e só então testa a condição de encerramento. Assim, o conteúdo acontece uma ou mais vezes. Atenção: a condição de `ate` significa PARAR quando verdadeira.

Primeira volta
O bloco sempre roda pelo menos uma vez.

Parada
A condição descreve quando terminar, não quando continuar.

Atualização
Se nada alterar a condição, o laço pode não terminar.

TERMINAL VISUALG — MISSAO 32

```
repita
escreval("1 - Continuar 0 - Sair")
leia(opcao)
ate opcao = 0
```

DESAFIO — +30 XP

Leia uma senha pelo menos uma vez e repita até a pessoa digitar 1234.

PISTA / GABARITO — A condição de parada é `ate senha = 1234`.

MUNDO 3 · 33

Contador e acumulador

Contador acompanha quantas ocorrências aconteceram; acumulador soma valores ao longo das voltas. Ambos precisam começar com um valor antes do laço.

Contador
Costuma receber +1 a cada evento ou volta.

Acumulador
Recebe acumulador + valor novo, como total <- total + compra.

Inicialização
Começar um total em 0 e um produto em 1 evita carregar lixo de outra missão.

TERMINAL VISUALG — MISSAO 33

```
total <- 0
para item de 1 ate 3 faca
leia(preco)
total <- total + preco
fimpara
```

DESAFIO — +30 XP

Some as idades de quatro pessoas e mostre o total.

PISTA / GABARITO — Use um inteiro total iniciado em 0 e um leia dentro do para.

MUNDO 3 · 34

Laços dentro de laços

Um laço pode existir dentro de outro. O laço externo escolhe a linha ou rodada; o interno executa as colunas ou tentativas de cada linha. Cada volta externa reinicia o percurso interno.

Custo

Se o externo repete 3 vezes e o interno 4, o bloco interno executa 12 vezes.

Indente

A indentação deixa visível qual fimpara fecha cada laço.

Use com intenção

Tabuadas, grades e combinações pequenas são bons exemplos; não aninhe sem necessidade.

TERMINAL VISUALG — MISSAO 34

```
para linha de 1 ate 2 faca
para coluna de 1 ate 3 faca
escreva("* " )
fimpara
escreval("")
fimpara
```

DESAFIO — +30 XP

Quantas estrelas o exemplo imprime no total?

PISTA / GABARITO — 2 linhas vezes 3 colunas: 6 estrelas.

MUNDO 3 · 35

Rastreie a execução

Rastrear é simular o algoritmo à mão, linha a linha, anotando o valor de cada variável e o resultado das condições. Uma tabela de rastreamento transforma uma surpresa em sequência observável.

Colunas
Inclua etapa, variável, condição e saída relevante.

Uma volta por vez
Não pule a atualização do contador nem avalie a condição depois da linha errada.

Previsão
Antes de executar, escreva o resultado esperado; depois compare com o resultado real.

TERMINAL VISUALG — MISSAO 35

```
x <- 2
para i de 1 ate 3 faca
x <- x + i
fimpara
```

DESAFIO — +30 XP

Rastreie x e i após cada volta.

PISTA / GABARITO — x termina em 8: 2+1=3, +2=5, +3=8.

MUNDO 3 · 36

Caça aos bugs

Bug é um defeito que faz o programa falhar, parar ou produzir resultado incorreto. Depurar é investigar com método, em vez de mudar várias linhas ao acaso.

Sintaxe
Uma palavra incorreta, bloco não fechado ou operador ausente impede interpretar o código.

Execução
Uma entrada inesperada pode revelar uma divisão por zero ou laço que nunca termina.

Lógica
O programa roda, mas a regra está errada: por exemplo, calcular desconto somando em vez de subtrair.

DESAFIO +30 XP

Leia `media <- nota1 + nota2 / 2`. O erro é de sintaxe ou de lógica matemática?

PISTA / GABARITO É lógico: faltam parênteses em torno da soma.

MUNDO 3 · 37

Teste limites e casos especiais

Testar só o caso comum não basta. Dados de fronteira e situações especiais ajudam a revelar erros que ficam escondidos no caminho mais fácil.

Limite inferior
Se a nota começa em 0, teste 0 e também um valor menor.

Limite superior
Se a faixa termina em 10, teste 10 e também um valor maior.

Caso vazio
Listas vazias, texto em branco e tentativa de dividir por zero precisam de uma regra.

DESAFIO +30 XP

Planeje quatro testes para um programa que diz se um número está entre 1 e 100.

PISTA / GABARITO Teste 1, 100, 0 e 101.

MUNDO 4 · 38

Divida em procedimentos

Procedimento nomeia uma tarefa reutilizável e executa comandos sem devolver um resultado como valor. No VisuAlg, declare procedimentos antes do `inicio` principal. Eles deixam o algoritmo mais organizado.

Uma tarefa

Cada procedimento deve ter uma responsabilidade fácil de explicar.

Chamada

Escreva o nome do procedimento no ponto em que quer executar a tarefa.

Parâmetros

Passe informações entre parênteses quando a tarefa precisar de dados diferentes a cada chamada.

TERMINAL VISUALG — MISSAO 38

```
procedimento mostrar_linha
inicio
escreval("----------------")
fimprocedimento

// no inicio principal:
mostrar_linha
```

DESAFIO — +30 XP

Crie um procedimento que mostre o título de um jogo e chame-o duas vezes.

PISTA / GABARITO — Sem parâmetros, no VisuAlg 3 não use parênteses na declaração ou chamada.

MUNDO 4 · 39

Funções devolvem um valor

Função executa uma tarefa e retorna um resultado que pode ser guardado numa variável ou usado numa expressão. A declaração informa o tipo do retorno, e `retorne` entrega o valor.

Entrada opcional
Parâmetros representam dados recebidos pela função.

Saída tipada
Uma função que devolve inteiro declara `: inteiro`; uma que devolve lógico declara `: logico`.

Reutilize
Uma regra em função pode ser chamada em vários pontos sem copiar o cálculo.

TERMINAL VISUALG — MISSAO 39

```
funcao dobro(n: inteiro): inteiro
inicio
retorne n * 2
fimfuncao

// uso: resultado <- dobro(6)
```

DESAFIO — +35 XP

Escreva a assinatura e o retorno de uma função que calcula o quadrado de um inteiro.

PISTA / GABARITO — `funcao quadrado(n: inteiro): inteiro`; retorne n * n.

MUNDO 4 · 40

Parâmetros e escopo

Parâmetro é um nome que recebe um dado na chamada. Variável local existe dentro de uma rotina; variável global é declarada fora e pode ser acessada em outros blocos. Prefira passar o que a rotina precisa de forma explícita.

Parâmetro por valor
A rotina recebe uma cópia do valor para usar no próprio cálculo.

Local
Uma variável declarada dentro da rotina serve àquela tarefa.

Global
Pode ser acessada por várias rotinas, mas mudanças inesperadas dificultam rastrear o estado.

TERMINAL VISUALG — MISSAO 40

```
funcao somar(a, b: inteiro): inteiro
inicio
retorne a + b
fimfuncao

resultado <- somar(4, 7)
```

DESAFIO — +30 XP

Quais valores a e b recebem na chamada somar(4, 7)?

PISTA / GABARITO — a recebe 4; b recebe 7; a função devolve 11.

MUNDO 4 · 41

Vetores: várias posições

Vetor guarda vários valores do mesmo tipo sob um nome e índices. É útil quando há notas de uma turma ou itens de uma lista. No VisuAlg, a faixa de índices faz parte da declaração.

Mesmo tipo
Um vetor de inteiros não mistura texto e números na mesma estrutura.

Índice
A posição identifica onde o valor fica; use índices dentro dos limites declarados.

Faixa
Com [1..5], as posições válidas são 1, 2, 3, 4 e 5.

TERMINAL VISUALG — MISSAO 41

```
var notas: vetor[1..3] de real
inicio
notas[1] <- 8.5
notas[2] <- 7.0
notas[3] <- 9.0
```

DESAFIO — +30 XP

Quantas posições há em `itens: vetor[0..4] de inteiro`?

PISTA / GABARITO — Cinco posições: 0, 1, 2, 3 e 4.

MUNDO 4 · 42

Percorra e resuma um vetor

Use um laço para visitar as posições de um vetor. No mesmo percurso, você pode contar valores, acumular uma soma e depois calcular uma média.

Índice de controle
O contador do laço deve coincidir com os índices válidos.

Acumulador
Comece soma em 0 antes de percorrer o vetor.

Média
Divida a soma pela quantidade, que precisa ser maior que zero.

TERMINAL VISUALG — MISSAO 42

```
soma <- 0
para i de 1 ate 3 faca
soma <- soma + notas[i]
fimpara
media <- soma / 3
```

DESAFIO — +35 XP

Para notas 6, 8 e 10, qual média o trecho calcula?

PISTA / GABARITO — A soma é 24; a média é 8.

MUNDO 4 · 43

Encontre o maior e procure um item

Busca linear verifica posições uma por uma até encontrar o valor ou terminar o vetor. Para achar o maior, compare cada item com o melhor valor conhecido até aquele momento.

Busca
Comece com encontrado falso; quando achar, marque verdadeiro.

Maior
Inicialize o maior com a primeira posição, não com zero se os dados podem ser negativos.

Posição
Guarde o índice encontrado se o programa precisar dizer onde o valor está.

TERMINAL VISUALG — MISSAO 43

```
encontrado <- falso
para i de 1 ate quantidade faca
se valores[i] = procurado entao
encontrado <- verdadeiro
fimse
fimpara
```

DESAFIO — +35 XP

Por que o maior deve começar em valores[1] e não em zero?

PISTA / GABARITO — Se todos os números forem negativos, zero não pertence à lista e pareceria o maior.

MUNDO 4 · 44

Matrizes: linhas e colunas

Matriz organiza valores em duas dimensões, como uma tabela de assentos ou um tabuleiro. Acessar uma célula exige dois índices: linha e coluna.

Declaração
VisuAlg permite vetor com dois intervalos, como matriz[1..3,1..4].

Acesso
Use matriz[linha,coluna] para localizar uma célula.

Percurso
Dois laços aninhados percorrem todas as linhas e colunas.

TERMINAL VISUALG — MISSAO 44

```
var tabuleiro: vetor[1..2,1..3] de caractere

para linha de 1 ate 2 faca
para coluna de 1 ate 3 faca
escreva(tabuleiro[linha,coluna])
fimpara
fimpara
```

DESAFIO — +35 XP

Quantas células existem em uma matriz 3 por 4?

PISTA / GABARITO — 3 x 4 = 12.

MUNDO 4 · 45

Textos também são dados

Nomes, comandos e respostas digitadas são dados caractere. Planeje o que o algoritmo deve fazer com espaços, letras maiúsculas e entradas inesperadas. Não trate um texto como número sem convertê-lo de forma adequada.

Mensagem
Use aspas duplas para textos literais.

Comparar
Uma opção como `resposta = "sim"` depende da forma exata digitada.

Orientar
Explique exemplos válidos na pergunta para reduzir respostas ambíguas.

TERMINAL VISUALG — MISSAO 45

```
var resposta: caractere
inicio
escreva("Deseja jogar? (sim/nao): " )
leia(resposta)
se resposta = "sim" entao
escreval("Vamos!")
fimse
```

DESAFIO — +30 XP

Que outra resposta precisa ser tratada para a pessoa que digita “não”?

PISTA / GABARITO — Crie um senao que encerre ou explique como começar.

CHEFE · 46

Mini game: adivinhe o código

Vamos combinar entrada, tentativa, contador, decisão e repetição. Uma pessoa escolhe um número secreto e a outra tenta adivinhar. Nesta versão de dois jogadores, quem prepara o jogo escolhe o segredo no código antes de entregar o teclado.

Estado

Guarde segredo, tentativa e quantidade de palpites.

Regra

Repita enquanto tentativa for diferente do segredo.

Feedback

Diga se o palpite é maior ou menor para orientar a próxima tentativa.

TERMINAL VISUALG — MISSAO 46

```
algoritmo "adivinhe"
var segredo, tentativa, tentativas: inteiro
inicio
segredo <- 7 // altere antes de jogar
tentativas <- 0
repita
escreva("Seu palpite: " )
leia(tentativa)
tentativas <- tentativas + 1
se tentativa < segredo entao
escreval("Maior")
senao
se tentativa > segredo entao
escreval("Menor")
fimse
fimse
ate tentativa = segredo
escreval("Acertou em ", tentativas, " tentativas!")
fimalgoritmo
```

DESAFIO — **+40 XP**

Adicione uma mensagem final com o número de tentativas usadas.

PISTA / GABARITO — O código já mostra tentativas; personalize uma regra ou faixa válida.

CHEFE · 47

Mini game: quiz de três perguntas

Um quiz calcula uma pontuação a partir de respostas. Cada pergunta compara a resposta com a correta e soma pontos quando há acerto. Teste uma sequência de tudo certo, tudo errado e respostas misturadas.

Comece em zero

A pontuação é acumulada, então precisa de uma inicialização.

Decida por resposta

Cada acerto altera o placar; um erro pode apenas mostrar a resposta correta.

Mostre o resultado

Dê contexto ao placar e use uma condição para classificar a conquista.

TERMINAL VISUALG — MISSAO 47

```
pontos <- 0
leia(resposta)
se resposta = "A" entao
pontos <- pontos + 1
fimse
// repita a regra para cada pergunta
```

DESAFIO — +40 XP

Se há três perguntas e a pessoa acerta duas, qual deve ser o placar?

PISTA / GABARITO — 2 pontos. Uma rotina de pergunta pode evitar copiar código.

PROJETO · 48

Planeje antes de codificar

Um projeto final junta ferramentas conhecidas. Antes do VisuAlg, escreva o objetivo, identifique dados, decomponha a solução, escolha variáveis e planeje testes. Esse roteiro diminui retrabalho.

Objetivo
O programa vai conduzir um quiz de matemática e informar a pontuação.

Dados
Pergunta e resposta são texto; pontos e rodada são inteiros.

Regras
Cada acerto soma 1; ao final, o programa mostra a faixa de desempenho.

Testes
Planeje uma execução com 0, 1 e todas as respostas corretas.

DESAFIO +35 XP

Escreva seu plano em cinco linhas antes de montar o programa da próxima página.

PISTA / GABARITO Se você não sabe como testar uma regra, ainda precisa torná-la mais clara.

PROJETO · 49

Projeto final: quiz de matemática

Este programa usa três perguntas, pontuação, entradas, decisões e uma classificação final. Digite o código, execute e rastreie uma partida. Depois personalize perguntas e mensagens sem mudar a estrutura da solução.

Observe
A pontuação começa em zero e só cresce em respostas corretas.

Rastreie
Depois de cada questão, confira resposta e pontos.

Expanda
Uma nova pergunta pede mais uma leitura e uma regra de pontuação.

TERMINAL VISUALG — MISSAO 49

```
algoritmo "quiz"
var r: caractere
pontos: inteiro
inicio
pontos <- 0
escreval("2 + 2?")
leia(r)
se r = "4" entao
pontos <- pontos + 1
fimse
escreval("Pontos: ", pontos)
fimalgoritmo
```

DESAFIO — +50 XP

Inclua mais duas perguntas e classifique: 0-1 precisa treinar; 2-3 mestre do quiz.

PISTA / GABARITO — Use comparações com pontos e `e` para construir a faixa.

EPÍLOGO · 50

Você desbloqueou o próximo nível

Você percorreu o caminho entre uma pergunta e um programa que pode ser executado. Guarde seus algoritmos, releia o que ainda parece confuso e refaça os chefes sem olhar as pistas. Programação cresce com prática deliberada.

Vocabulário

Algoritmo: passos finitos. Variável: nome associado a um valor. Condição: pergunta lógica. Laço: repetição controlada.

Próxima trilha

Aprenda a modelar problemas maiores, pratique estruturas de dados e depois transfira a lógica para uma linguagem de uso profissional.

Seu portfólio

Salve o quiz, o jogo de adivinhação e uma solução própria com enunciado, código e casos de teste.

Consulta

Para conferir sintaxe e recursos, consulte o Manual VisuAlg da EEL-USP e os materiais de aprendizagem de VisuAlg Web.

DESAFIO **+20 XP**

Escreva uma frase: “Agora consigo explicar algoritmo como...”. Depois escolha seu próximo desafio.

PISTA / GABARITO

Parabéns pela conclusão! XP conquistados, conhecimento desbloqueado. Sua curiosidade é o melhor equipamento. FIM DA FASE 1.